

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Recurso) MEIC, MEIM	
Nº	Nome:	01/02/2023

EXAME ÉPOCA RECURSO (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (—). Uma opção assinalada corretamente soma 0,25 valores e incorretamente desconta 0,125 valores (50% da cotação de cada alínea).

1. [1 val.] Sobre as características dos sistemas distribuídos

☐	O teorema CAP demonstra que num sistema distribuído, temos sempre 100% das características de consistência (C), disponibilidade (A) e partições (P) por existência de falhas
☐	A característica de <i>Speedup</i> de um sistema permite definir o desempenho em termos de capacidade de CPU
☐	A disponibilidade de um sistema é definida pela razão, em percentagem, do tempo em que um sistema funciona (<i>Uptime</i>) versus o tempo em que o sistema deveria funcionar (<i>Uptime+Downtime</i>)
☐	A Latência de comunicação de dados em aplicações distribuídas é sempre maior que zero

2. [1 val.] Sobre o conceito de Tempo e Coordenação em sistemas distribuídos

☐	Os valores dos relógios físicos (hh:mm:ss) dos múltiplos computadores podem ser sincronizados usando o algoritmo <i>Cristian</i> a partir de um servidor central com acesso a um relógio de grande precisão
☐	Em sistemas de comunicação <i>multicast</i> , os relógios lógicos vetoriais transportam nas mensagens valores dos relógios entre os participantes que permitem detetar na receção falta de ordem nas mensagens
☐	Os relógios lógicos como proposto por Leslie Lamport em 1978 permitem ordenar os eventos num sistema distribuído através de um participante coordenador que é eleito com o algoritmo <i>Bully</i>
☐	A causalidade entre eventos é fundamental na medida em que se existem duas mensagens A e B, em que B é consequência de A, então deve garantir-se que todos os participantes recebem primeiro A e depois B

3. [1 val.] Relativamente à operação gRPC: `rpc oper(stream Msg1) returns (Msg2)`

☐	O cliente pode chamar <code>onNext()</code> num <i>stream</i> para enviar N mensagens do tipo <code>Msg1</code> , terminando com a chamada de <code>onCompleted()</code> . O servidor, então responde com única mensagem do tipo <code>Msg2</code>
☐	A operação <code>oper</code> pode ser chamada nos clientes com um <i>stub</i> bloqueante ou com um <i>stub</i> não bloqueante
☐	Em <i>protobuf</i> a mensagem <code>Msg2</code> pode ser vazia, sendo definida como: <code>message Msg2 { }</code>
☐	Num cliente, a chamada da operação <code>oper</code> pode ser feita pelo seguinte código: <pre>StreamObserver<Msg2> str2= new someStreamObserver<Msg2>() { . . . }; StreamObserver<Msg1> str1=stubNonBlocked.oper(str2);</pre>

4. [1 val.] Sobre o sistema RabbitMQ

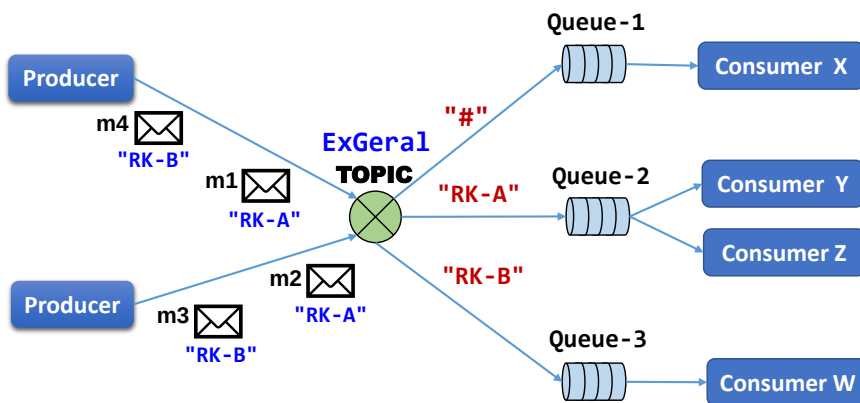
☐	Um <i>Exchange</i> do tipo TOPIC é mais genérico do que um do tipo DIRECT, pois tem <i>routing keys</i> flexíveis usando os caracteres * e # para que as filas (<i>queues</i>) recebam mensagens com diferentes padrões de encaminhamento
☐	Se uma fila (<i>queue</i>) tiver associação com múltiplos consumidores todos eles recebem as mesmas mensagens
☐	Quando um consumidor dá <i>acknowledge</i> negativo a uma mensagem o produtor da mensagem recebe automaticamente uma notificação
☐	Quando é enviada uma mensagem M para um <i>exchange</i> do tipo FANOUT, associado a N filas (<i>queues</i>) em que cada <i>queue</i> tem associado um único consumidor, então todos os consumidores recebem a mensagem M

5. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consensos

☐	Para resolver os conflitos de acesso a um recurso crítico, o algoritmo de exclusão mútua <i>Ricart&Agrawala</i> utiliza a relação de precedência (Aconteceu-Antes) proporcionada pelos relógios lógicos de Lamport
☐	O algoritmo de exclusão mútua com topologia em anel, não usa relógios lógicos de Lamport mas também garante uma relação de precedência (Aconteceu-Antes)
☐	O protocolo <i>Two-phase commit (2PC)</i> assume que não há falhas do coordenador ou de algum participante durante as fases de <i>Voting phase</i> e de <i>Commit phase</i>
☐	No algoritmo <i>Raft</i> os clientes submetem sempre os pedidos a um participante eleito como líder, por um algoritmo de consenso entre todos os participantes, que garante a replicação consistente nos outros participantes

6. [2 val.] Considere um sistema distribuído, constituído por 3 participantes (**p0**, **p1**, **p2**), que utiliza comunicação por *multicast* fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Quando **p0 envia a mensagem** com o vetor **[6,2,6]**, justifique o que acontece na atualização do vetor local nos outros participantes p1 e p2, considerando que:
- a) p1 tem no seu vetor local **p1=[5,3,6]**; b) p2 tem no seu vetor local **p2=[4,1,6]**.

7. [2 val.] Considere um sistema desenvolvido com o *middleware* RabbitMQ que usa o *exchange* **ExGeral** do tipo TOPIC para que diferentes produtores enviem mensagens. Associadas (*binding*) ao *exchange* **ExGeral** existem 3 *queues* com as *routing keys* (“#”, “RK-A” e “RK-B”). Considerando o envio das mensagens m1, m2, m3 e m4 e respectivas *routing keys* indicadas, escreva sobre o diagrama da figura o fluxo de encaminhamento dessas mensagens, indicando claramente à frente de cada consumidor as mensagens que cada um receberá.



8. [2 val.] Um mau programador implementou uma aplicação servidora (web) colocando *hard-coded* o porto TCP/IP 8080, onde a aplicação escuta os pedidos HTTP, vindos das aplicações cliente. Apresente os passos principais para colocar múltiplos servidores web em execução, em diferentes portos TCP/IP, numa única máquina física Linux (computador *host*), considerando que essa máquina já tem instalado o sistema de virtualização *Docker*.